PostgreSQL Data Upload Using Pandas and SQLAlchemy

Assignment: Upload Formula 1 dataset to the college PostgreSQL database
Target Host: 10.100.71.21
Target Database / Schema: 202618013 / DBMS_project
Source Data: Formula 1 World Championship (1950–2024)
Due Date: 19 October 2026

Objective

1. Importing the CSV files into pandas DataFrames
2. Creating the SQLAlchemy connection and the `DBMS_project` schema
3. Creating the DDL (tables, constraints and indexes)
4. Uploading the data into the tables
5. Checking the uploaded data and closing the connection

Tables

- circuits
- constructors
- drivers
- status
- races
- qualifying
- results
- sprint_results
- lap_times
- pit_stops
- driver_standings
- constructor_standings

Team Members

1. 202618016 - Krishika Lalwani
2. 202618013 - Devanshi vora
3. 202618052 - Lakshita Pagaria

In [77]:
```
import pandas as pd
from sqlalchemy import create_engine, URL, text
```

In [78]:
```
circuits = pd.read_csv("df/circuits.csv")
constructor_standings = pd.read_csv("df/constructor_standings.csv")
constructors = pd.read_csv("df/constructors.csv")
driver_standings = pd.read_csv("df/driver_standings.csv")
drivers = pd.read_csv("df/drivers.csv")
lap_times = pd.read_csv("df/lap_times.csv")
pit_stops = pd.read_csv("df/pit_stops.csv")
qualifying = pd.read_csv("df/qualifying.csv")
races = pd.read_csv("df/races.csv")
results = pd.read_csv("df/results.csv")
sprint_results = pd.read_csv("df/sprint_results.csv")
status = pd.read_csv("df/status.csv")
```

Create the SQLAlchemy connection

In [79]:
```
username = "202618013"
password = "202618013"
host = "10.100.71.21"
port = 5432
database = "202618013"
schema = "DBMS_project"

url = URL.create(
    "postgresql+psycopg",
    username=username,
    password=password,
    host=host,
    port=port,
    database=database
)

# Create the engine
engine = create_engine(
    url,
    connect_args={
        "options": f"-csearch_path={schema},public"
    }
)

# Create the schema
with engine.begin() as connection:
    connection.execute(
        text(f'CREATE SCHEMA IF NOT EXISTS "{schema}"')
    )

print(f"Connected to PostgreSQL database '{database}'")
print(f"Using schema '{schema}'")
```

```
Connected to PostgreSQL database '202618013'
Using schema 'DBMS_project'
```

In [80]:
```
try:
    with engine.connect() as connection:
        result = connection.execute(text("SELECT version();"))
        print("PostgreSQL connection successful!")
        print(result.scalar())
except Exception as e:
    print("Connection failed:")
    print(e)
```

```
PostgreSQL connection successful!
PostgreSQL 10.20 on x86_64-pc-linux-gnu, compiled by gcc (GCC) 4.8.5
20150623 (Red Hat 4.8.5-44), 64-bit
```

Create the DDL

In [81]:
```
ddl_script = r'''
CREATE TABLE IF NOT EXISTS circuits (
    "circuitId"   INT           PRIMARY KEY,
    "circuitRef"  VARCHAR(50)   UNIQUE NOT NULL,
    name          VARCHAR(100)  NOT NULL,
    location      VARCHAR(100),
    country       VARCHAR(60)
);

CREATE TABLE IF NOT EXISTS constructors (
    "constructorId"   INT          PRIMARY KEY,
    "constructorRef"  VARCHAR(50)  UNIQUE NOT NULL,
    name              VARCHAR(100) NOT NULL,
    nationality       VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS drivers (
    "driverId"    INT          PRIMARY KEY,
    "driverRef"   VARCHAR(50)  UNIQUE NOT NULL,
    forename      VARCHAR(50)  NOT NULL,
    surname       VARCHAR(50)  NOT NULL,
    dob           DATE,
    nationality   VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS status (
    "statusId" INT           PRIMARY KEY,
    status     VARCHAR(100)  NOT NULL
);

CREATE TABLE IF NOT EXISTS races (
    "raceId"     INT          PRIMARY KEY,
    year       SMALLINT     NOT NULL,
    round      SMALLINT     NOT NULL,
    name       VARCHAR(100) NOT NULL,
    date       DATE         NOT NULL,
    time       TIME         NULL,
```

```
    "circuitId"  INT          NOT NULL,

    CONSTRAINT uq_races_year_rnd UNIQUE (year, round),

    CONSTRAINT fk_races_circuit
        FOREIGN KEY ("circuitId")
        REFERENCES circuits ("circuitId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_races_year
        CHECK (year BETWEEN 1950 AND 2100),

    CONSTRAINT ck_races_round
        CHECK (round > 0)
);

CREATE TABLE IF NOT EXISTS results (
    "resultId"        INT            PRIMARY KEY,
    "raceId"          INT            NOT NULL,
    "driverId"        INT            NOT NULL,
    "constructorId"   INT            NOT NULL,
    "statusId"        INT            NOT NULL,
    points            DECIMAL(5,2)   NOT NULL DEFAULT 0,
    position          SMALLINT       NULL,
    grid              SMALLINT       NOT NULL DEFAULT 0,
    laps              SMALLINT       NOT NULL DEFAULT 0,
    rank              SMALLINT       NULL,
    "positionOrder"   SMALLINT       NOT NULL,
    "fastestLapTime"  VARCHAR(12)    NULL,

    CONSTRAINT uq_results_race_driver
        UNIQUE ("raceId", "driverId"),

    CONSTRAINT fk_results_race
        FOREIGN KEY ("raceId")
        REFERENCES races ("raceId")
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT fk_results_driver
        FOREIGN KEY ("driverId")
        REFERENCES drivers ("driverId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT fk_results_constructor
        FOREIGN KEY ("constructorId")
        REFERENCES constructors ("constructorId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,
```

```
    CONSTRAINT fk_results_status
        FOREIGN KEY ("statusId")
        REFERENCES status ("statusId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_results_points
        CHECK (points ⩾ 0),

    CONSTRAINT ck_results_grid
        CHECK (grid ⩾ 0),

    CONSTRAINT ck_results_laps
        CHECK (laps ⩾ 0)
);

CREATE TABLE IF NOT EXISTS qualifying (
    "qualifyId"       INT          PRIMARY KEY,
    "raceId"          INT          NOT NULL,
    "driverId"        INT          NOT NULL,
    "constructorId"   INT          NOT NULL,
    position          SMALLINT     NOT NULL,

    CONSTRAINT uq_qualifying_race_driver
        UNIQUE ("raceId", "driverId"),

    CONSTRAINT fk_qualifying_race
        FOREIGN KEY ("raceId")
        REFERENCES races ("raceId")
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT fk_qualifying_driver
        FOREIGN KEY ("driverId")
        REFERENCES drivers ("driverId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT fk_qualifying_constructor
        FOREIGN KEY ("constructorId")
        REFERENCES constructors ("constructorId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_qualifying_pos
        CHECK (position IS NULL OR position > 0)
);

CREATE TABLE IF NOT EXISTS sprint_results (
    "resultId"        INT            PRIMARY KEY,
    "raceId"          INT            NOT NULL,
    "driverId"        INT            NOT NULL,
```

```
    "statusId"        INT            NOT NULL,
    "constructorId"   INT            NOT NULL,
    points            DECIMAL(5,2)   NOT NULL DEFAULT 0,
    position          SMALLINT       NULL,
    "positionOrder"   SMALLINT       NOT NULL,
    grid              SMALLINT       NOT NULL DEFAULT 0,
    laps              SMALLINT       NOT NULL DEFAULT 0,

    CONSTRAINT uq_sprint_race_driver
        UNIQUE ("raceId", "driverId"),

    CONSTRAINT fk_sprint_race
        FOREIGN KEY ("raceId")
        REFERENCES races ("raceId")
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT fk_sprint_driver
        FOREIGN KEY ("driverId")
        REFERENCES drivers ("driverId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT fk_sprint_status
        FOREIGN KEY ("statusId")
        REFERENCES status ("statusId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT fk_sprint_constructor
        FOREIGN KEY ("constructorId")
        REFERENCES constructors ("constructorId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_sprint_points
        CHECK (points >= 0)
);

CREATE TABLE IF NOT EXISTS lap_times (
    "raceId"        INT          NOT NULL,
    "driverId"      INT          NOT NULL,
    lap             SMALLINT     NOT NULL,
    milliseconds    INT          NOT NULL,

    CONSTRAINT pk_lap_times
        PRIMARY KEY ("raceId", "driverId", lap),

    CONSTRAINT fk_laptimes_race
        FOREIGN KEY ("raceId")
        REFERENCES races ("raceId")
        ON UPDATE CASCADE
```

```
        ON DELETE CASCADE,

    CONSTRAINT fk_laptimes_driver
        FOREIGN KEY ("driverId")
        REFERENCES drivers ("driverId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_laptimes_lap
        CHECK (lap > 0),

    CONSTRAINT ck_laptimes_ms
        CHECK (milliseconds > 0)
);

CREATE TABLE IF NOT EXISTS pit_stops (
    "raceId"        INT          NOT NULL,
    "driverId"      INT          NOT NULL,
    stop            SMALLINT     NOT NULL,
    lap             SMALLINT     NOT NULL,
    milliseconds    INT          NOT NULL,

    CONSTRAINT pk_pit_stops
        PRIMARY KEY ("raceId", "driverId", stop),

    CONSTRAINT fk_pitstops_race
        FOREIGN KEY ("raceId")
        REFERENCES races ("raceId")
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT fk_pitstops_driver
        FOREIGN KEY ("driverId")
        REFERENCES drivers ("driverId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_pitstops_stop
        CHECK (stop > 0),

    CONSTRAINT ck_pitstops_lap
        CHECK (lap > 0),

    CONSTRAINT ck_pitstops_ms
        CHECK (milliseconds > 0)
);

CREATE TABLE IF NOT EXISTS driver_standings (
    "driverStandingsId"  INT           PRIMARY KEY,
    "raceId"             INT           NOT NULL,
    "driverId"           INT           NOT NULL,
    points               DECIMAL(6,2)  NOT NULL DEFAULT 0,
```

```
    position             SMALLINT      NOT NULL,
    wins                 SMALLINT      NOT NULL DEFAULT 0,

    CONSTRAINT uq_dstand_race_driver
        UNIQUE ("raceId", "driverId"),

    CONSTRAINT fk_dstand_race
        FOREIGN KEY ("raceId")
        REFERENCES races ("raceId")
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT fk_dstand_driver
        FOREIGN KEY ("driverId")
        REFERENCES drivers ("driverId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_dstand_points
        CHECK (points >= 0),

    CONSTRAINT ck_dstand_wins
        CHECK (wins >= 0)
);

CREATE TABLE IF NOT EXISTS constructor_standings (
    "constructorStandingsId"  INT           PRIMARY KEY,
    "raceId"                  INT           NOT NULL,
    "constructorId"           INT           NOT NULL,
    points                    DECIMAL(6,2)  NOT NULL DEFAULT 0,
    position                  SMALLINT      NOT NULL,
    wins                      SMALLINT      NOT NULL DEFAULT 0,

    CONSTRAINT uq_cstand_race_constr
        UNIQUE ("raceId", "constructorId"),

    CONSTRAINT fk_cstand_race
        FOREIGN KEY ("raceId")
        REFERENCES races ("raceId")
        ON UPDATE CASCADE
        ON DELETE CASCADE,

    CONSTRAINT fk_cstand_constructor
        FOREIGN KEY ("constructorId")
        REFERENCES constructors ("constructorId")
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_cstand_points
        CHECK (points >= 0),

    CONSTRAINT ck_cstand_wins
```

```
        CHECK (wins >= 0)
);

CREATE INDEX IF NOT EXISTS idx_races_year
    ON races (year);

CREATE INDEX IF NOT EXISTS idx_races_circuit
    ON races ("circuitId");

CREATE INDEX IF NOT EXISTS idx_results_driver
    ON results ("driverId");

CREATE INDEX IF NOT EXISTS idx_results_constructor
    ON results ("constructorId");

CREATE INDEX IF NOT EXISTS idx_results_status
    ON results ("statusId");

CREATE INDEX IF NOT EXISTS idx_results_rank
    ON results ("rank");

CREATE INDEX IF NOT EXISTS idx_qualifying_driver
    ON qualifying ("driverId");

CREATE INDEX IF NOT EXISTS idx_laptimes_driver
    ON lap_times ("driverId");

CREATE INDEX IF NOT EXISTS idx_pitstops_driver
    ON pit_stops ("driverId");

CREATE INDEX IF NOT EXISTS idx_dstand_driver
    ON driver_standings ("driverId");

CREATE INDEX IF NOT EXISTS idx_cstand_constructor
    ON constructor_standings ("constructorId");

CREATE INDEX IF NOT EXISTS idx_sprint_driver
    ON sprint_results ("driverId");

CREATE INDEX IF NOT EXISTS idx_drivers_nationality
    ON drivers (nationality);

CREATE INDEX IF NOT EXISTS idx_circuits_country
    ON circuits (country);
'''

# Split the DDL into individual SQL statements
ddl_statements = [
    statement.strip()
    for statement in ddl_script.split(';')
    if statement.strip()
]
```

```
# Execute each statement
with engine.begin() as connection:
    for statement in ddl_statements:
        connection.exec_driver_sql(statement)

print("Skeleton creation completed: tables and indexes are ready.")
```

```
Skeleton creation completed: tables and indexes are ready.
```

uploading the data into the table

In [82]:

```
tables = {
    "circuits": circuits,
    "constructor_standings": constructor_standings,
    "constructors": constructors,
    "driver_standings": driver_standings,
    "drivers": drivers,
    "lap_times": lap_times,
    "pit_stops": pit_stops,
    "qualifying": qualifying,
    "races": races,
    "results": results,
    "sprint_results": sprint_results,
    "status": status
}

table_columns = {
    "circuits": ["circuitId", "circuitRef", "name", "location", "country"],
    "constructors": ["constructorId", "constructorRef", "name",
"nationality"],
    "drivers": ["driverId", "driverRef", "forename", "surname", "dob",
"nationality"],
    "status": ["statusId", "status"],
    "races": ["raceId", "year", "round", "name", "date", "time",
"circuitId"],
    "results": ["resultId", "raceId", "driverId", "constructorId",
"statusId",
                "points", "position", "grid", "laps", "rank",
"positionOrder",
                "fastestLapTime"],
    "qualifying": ["qualifyId", "raceId", "driverId", "constructorId",
"position"],
    "sprint_results": ["resultId", "raceId", "driverId", "statusId",
"constructorId",
                       "points", "position", "positionOrder", "grid",
"laps"],
    "lap_times": ["raceId", "driverId", "lap", "milliseconds"],
    "pit_stops": ["raceId", "driverId", "stop", "lap", "milliseconds"],
    "driver_standings": ["driverStandingsId", "raceId", "driverId",
"points",
                         "position", "wins"],
```

```
    "constructor_standings": ["constructorStandingsId", "raceId",
"constructorId",
                              "points", "position", "wins"]
}

# Parent tables must be uploaded before tables that reference them through
foreign keys.
upload_order = [
    "circuits",
    "constructors",
    "drivers",
    "status",
    "races",
    "qualifying",
    "results",
    "sprint_results",
    "lap_times",
    "pit_stops",
    "driver_standings",
    "constructor_standings"
]

for table_name in upload_order:
    print(f"Uploading {table_name}...")

    df_to_upload = tables[table_name]
    required_columns = table_columns[table_name]

    missing_columns = [col for col in required_columns if col not in
df_to_upload.columns]
    if missing_columns:
        raise ValueError(
            f"{table_name}: these required columns were not found in the
CSV: {missing_columns}\n"
            f"Available columns: {df_to_upload.columns.tolist()}"
        )

    df_to_upload[required_columns].to_sql(
        name=table_name,
        con=engine,
        schema=schema,
        if_exists="append",
        index=False
    )

    print(f"{table_name} uploaded successfully ({len(df_to_upload)} rows).")
```

```
Uploading circuits...
circuits uploaded successfully (77 rows).
Uploading constructors...
constructors uploaded successfully (212 rows).
Uploading drivers...
```

```
drivers uploaded successfully (861 rows).
Uploading status...
status uploaded successfully (139 rows).
Uploading races...
races uploaded successfully (1125 rows).
Uploading qualifying...
qualifying uploaded successfully (10494 rows).
Uploading results...
results uploaded successfully (26759 rows).
Uploading sprint_results...
sprint_results uploaded successfully (360 rows).
Uploading lap_times...
lap_times uploaded successfully (589081 rows).
Uploading pit_stops...
pit_stops uploaded successfully (11371 rows).
Uploading driver_standings...
driver_standings uploaded successfully (34863 rows).
Uploading constructor_standings...
constructor_standings uploaded successfully (13391 rows).
```

check the uploaded data

In [83]:
```
with engine.connect() as connection:
    for table_name in tables:
        query = text(f'SELECT COUNT(*) FROM "{schema}"."{table_name}"')
        row_count = connection.execute(query).scalar()
        print(f"Table: {table_name} | Total rows: {row_count}")
```

```
Table: circuits | Total rows: 154
Table: constructor_standings | Total rows: 26782
Table: constructors | Total rows: 424
Table: driver_standings | Total rows: 69726
Table: drivers | Total rows: 1722
Table: lap_times | Total rows: 1178162
Table: pit_stops | Total rows: 22742
Table: qualifying | Total rows: 20988
Table: races | Total rows: 2250
Table: results | Total rows: 53518
Table: sprint_results | Total rows: 720
Table: status | Total rows: 278
```

In [84]:
```
engine.dispose()
print("Database connection closed.")
```

```
Database connection closed.
```